UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

FACULTAD DE INGENIERÍA

INGENIERÍA EN AUTOMATIZACIÓN

Control III

Construcción, instrumentación y control de un Péndulo Invertido Sobre Dos Ruedas Seguidor de Línea (PISDRSL)

Primer entrega

Profesor:
Dr. Roberto Valentín Carrillo Serrano

Equipo 1

Integrantes:

Aguilar Sarabia Diego Eugenio _____

Fernández Rendón Anna Patricia _____

Sánchez Alejo Frida Alexandra _____

Zeron Marin Luis Alejandro _____

Fecha compromiso: 5 de marzo del 2026
Fecha en que se entregó: 5 de marzo del 2026.

Índice

1. Objetivo **2**

2. Marco Teórico **2**
2.1. Modelo no lineal del péndulo con dos ruedas . . . 2
2.2. Modelo linealizado simplificado del péndulo con dos ruedas . . . 4
2.3. Controlador de seguimiento de línea . . . 4
2.4. Salida plana . . . 5
2.5. Controlador para balanceo y avance traslacional . . . 5

3. Materiales y Métodos **7**
3.1. Materiales . . . 7
3.2. Diagrama de conexiones entre componentes . . . 10
3.3. Metodología . . . 10

4. Resultados y Discusión **11**
4.1. Aguilar Sarabia Diego Eugenio . . . 11
4.2. Fernández Rendón Anna Patricia . . . 12
4.3. Sánchez Alejo Frida Alexandra . . . 12
4.4. Zeron Marin Luis Alejandro . . . 12
4.5. Resultados y discusión del equipo . . . 12

5. Conclusiones **12**
5.1. Aguilar Sarabia Diego Eugenio . . . 12
5.2. Fernández Rendón Anna Patricia . . . 12
5.3. Sánchez Alejo Frida Alexandra . . . 12
5.4. Zeron Marin Luis Alejandro . . . 12
5.5. Conclusiones del equipo . . . 12

Bibliografía **12**

6. Anexos **13**
6.1. Código Fuente del Sistema de Control de Equilibrio y Seguimiento de Línea en C (CCS Compiler) 13

Índice de figuras

1. Péndulo con dos ruedas. . . . 2
2. Diagrama de simulación analógica de (24) y (33). . . . 5
3. Diagrama de bloques de la Figura 2. . . . 5
4. Diagrama de bloques simplificado de la Figura 3. . . . 6
5. Diagrama de bloques del sistema en lazo cerrado. . . . 6
6. Diagrama de bloques del sistema en lazo cerrado con retroalimentación negativa, $U_n(s) = -U(s)$. 7
7. Conexión eléctrica de los componentes en el PISDRSL. . . . 10
8. Imagen de oscilocopio con un muestreo de 10 $[ms]$ y la señal tiene una frecuencia de 45 $[Hz]$ sin filtro. . . . 11

Índice de cuadros

1. Componentes electrónicos y mecánicos utilizados. . . . 8
2. Descripción de componentes. . . . 9
3. Equipo de cómputo utilizado. . . . 9
4. El software empleado para el desarrollo de la práctica. . . . 9

1. Objetivo

Construir, instrumentar y controlar de manera embebida un robot de péndulo invertido sobre dos ruedas seguidor de línea (PISDRSL), con capacidad para superar pendientes de 15 [°] y realizar trayectorias curvas (giros a la izquierda y derecha) siguiendo una línea blanca sobre un fondo negro. Para ello, se empleará un microcontrolador PIC18F4550 y comunicación inalámbrica vía Bluetooth con una estación base para la monitorización y graficación del comportamiento de las variables dinámicas del sistema durante su recorrido experimental en la pista.

2. Marco Teórico

2.1. Modelo no lineal del péndulo con dos ruedas

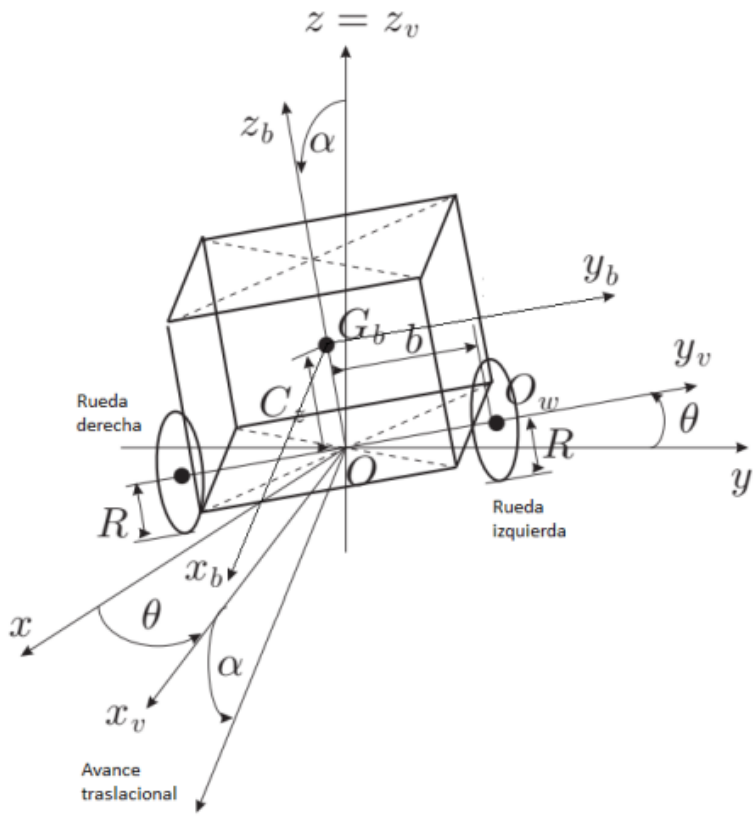

Figura 1: Péndulo con dos ruedas.

El modelo dinámico no lineal del péndulo con dos ruedas obtenido con las ecuaciones de Euler-Lagrange está dado por (**?**):

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = B(q)\tau \tag{1}$$

$$q = [x_o \quad y_o \quad \theta \quad \alpha \quad \phi_r \quad \phi_l]^T \tag{2}$$

$$M(q) = \begin{bmatrix} M_p(1+\tan^2(\beta)) & 0 & \Delta_1 & \Delta_2 & 0 & 0 \\ 0 & M_p & \Delta_3 & \Delta_4 & 0 & 0 \\ \Delta_1 & \Delta_3 & \Delta_5 & 0 & 0 & 0 \\ \Delta_2 & \Delta_4 & 0 & M_pC_z^2 + I_{yy} & 0 & 0 \\ 0 & 0 & 0 & 0 & M_wR^2 + I_{wa} & 0 \\ 0 & 0 & 0 & 0 & 0 & M_wR^2 + I_{wa} \end{bmatrix} \tag{3}$$

$$\Delta_1 = -M_p C_z \sin(\alpha)\sin(\theta) \tag{4}$$

$$\Delta_2 = M_p C_z(\cos(\alpha)\cos(\theta) - \sin(\alpha)\tan(\beta)) \tag{5}$$

$$\Delta_3 = M_p C_z \sin(\alpha)\cos(\theta) \tag{6}$$

$$\Delta_4 = M_p C_z \cos(\alpha)\sin(\theta) \tag{7}$$

$$\Delta_5 = (M_p C_z^2 + I_{xx})\sin^2(\alpha) + I_{zz}\cos^2(\alpha) + 2I_{wd} \tag{8}$$

$$C(q,\dot{q}) = \begin{bmatrix} 0 & 0 & \Gamma_1 & \Gamma_2 & 0 & 0 \\ 0 & 0 & \Gamma_3 & \Gamma_4 & 0 & 0 \\ 0 & 0 & \Gamma_5 & \Gamma_6 & 0 & 0 \\ 0 & 0 & -\Gamma_6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{9}$$

$$\Gamma_1 = -M_p C_z(\dot{\theta}\sin(\alpha)\cos(\theta) + \dot{\alpha}\sin(\theta)\cos(\alpha)) \tag{10}$$

$$\Gamma_2 = -M_p C_z(\dot{\theta}\sin(\theta)\cos(\alpha) + \dot{\alpha}\sin(\alpha)\cos(\theta) + \dot{\alpha}\cos(\alpha)\tan(\beta)) \tag{11}$$

$$\Gamma_3 = -M_p C_z(\dot{\theta}\sin(\alpha)\sin(\theta) - \dot{\alpha}\cos(\alpha)\cos(\theta)) \tag{12}$$

$$\Gamma_4 = M_p C_z(\dot{\theta}\cos(\alpha)\cos(\theta) - \dot{\alpha}\sin(\alpha)\sin(\theta)) \tag{13}$$

$$\Gamma_5 = (M_p C_z^2 + I_{xx} - I_{zz})\dot{\alpha}\sin(\alpha)\cos(\alpha) \tag{14}$$

$$\Gamma_6 = (M_p C_z^2 + I_{xx} - I_{zz})\dot{\theta}\sin(\alpha)\cos(\alpha) \tag{15}$$

$$g(q) = \begin{bmatrix} (M_p + 2M_w)g\tan(\beta) \\ 0 \\ 0 \\ -M_p C_z g\sin(\alpha) \\ 0 \\ 0 \end{bmatrix} \tag{16}$$

$$B(q) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ -1 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{17}$$

$$\tau = \begin{bmatrix} \tau_r \\ \tau_l \end{bmatrix} \tag{18}$$

donde las coordenadas del punto O son (x_o, y_o), β es el ángulo de una pendiente que sube el robot al avanzar, R es el radio de la rueda, $g = 9.81\ [\frac{m}{s^2}]$ es la aceleración de la gravedad, M_p es la masa del péndulo, C_z es la distancia del centro de masa del péndulo al punto O, b es la distancia del punto O al punto O_w, I_{xx} es el momento de inercia calculado alrededor del eje x_b, I_{yy} es el momento de inercia calculado alrededor del eje y_b, I_{zz} es el momento de inercia calculado alrededor del eje z_b, I_{wd} es el momento de inercia de la rueda alrededor de su diámetro, I_{wa} es el momento de inercia de la rueda alrededor de su eje, M_w es la masa completa de la

rueda (llanta, rin, cople y rotor), ϕ_r es el ángulo de rotación de la rueda derecha, ϕ_l es el ángulo de rotación de la rueda izquierda, α es el ángulo de inclinación del péndulo, θ es el ángulo de orientación del robot y v es la velocidad traslacional del robot dada como:

$$v = \frac{R}{2}(\dot{\phi}_r + \dot{\phi}_l) \tag{19}$$

La velocidad de orientación del péndulo con ruedas está dada por:

$$\dot{\theta} = \frac{R}{2b}(\dot{\phi}_r - \dot{\phi}_l) \tag{20}$$

2.2. Modelo linealizado simplificado del péndulo con dos ruedas

Cuando el robot realiza la tarea de seguidor de línea en una pista (línea de trayectoria deseada) las mediciones de las coordenadas (x_o, y_o) del punto O no son necesarias ya que esas variables se controlan indirectamente cuando la trayectoria deseada es seguida y esto mismo sucede con las posiciones de las ruedas del robot. Tomando ventaja de lo anteriormente descrito, es posible encontrar un modelo simplificado del robot.

Linealizando el modelo con restricciones no holonómicas del robot, asumiendo que $\beta = 0$, $\alpha \approx 0$ lo que implica $\cos(\alpha) \approx 1$, $\sin(\alpha) \approx \alpha$ y definiendo I_ϕ como (**?**):

$$I_\phi = M_w R^2 + I_{wa} \tag{21}$$

se encuentra la aceleración traslacional del robot como:

$$\boxed{I_\phi \dot{v} = \frac{R}{2}(\tau_r + \tau_l)} \tag{22}$$

donde τ_r es el par aplicado a la rueda derecha y τ_l es el par aplicado a la rueda izquierda.

Linealizando el modelo en (1) se tiene (**?**):

$$\boxed{[d_5 + I_\phi]\ddot{\theta} = \frac{R}{2b}(\tau_r - \tau_l)} \tag{23}$$

$$\boxed{\ddot{\alpha} - \frac{\sigma_1}{M_{22}}\alpha = -\sigma_2(\tau_r + \tau_l)} \tag{24}$$

$$d_5 = I_{zz} + 2I_{wd} \tag{25}$$

$$M_{22} = M_p C_z^2 + I_{yy} \tag{26}$$

$$\sigma_1 = g M_p C_z \tag{27}$$

$$\sigma_2 = \frac{1}{M_{22}}\left[\frac{M_p C_z R}{2I_\phi} + 1\right] \tag{28}$$

Así, el modelo linealizado simplificado del péndulo con dos ruedas está dado por las ecuaciones (22), (23) y (24).

2.3. Controlador de seguimiento de línea

Se define el controlador proporcional con retroalimentación de velocidad para seguimiento de línea (orientación del robot) de la siguiente manera (**?**)

$$\boxed{\tau_a = \tau_r - \tau_l = \frac{2b}{R}\left[-k_v\dot{\theta} - k_p\tilde{\theta}\right], \quad \tilde{\theta} = \theta - \theta_d} \tag{29}$$

donde θ_d es una constante y representa el valor deseado de la orientación del robot. Al sustituir (29) en (23) resulta:

$$[d_5 + I_\phi]\ddot{\theta} + k_v\dot{\theta} + k_p\theta = k_p\theta_d \tag{30}$$

Así, si los coeficientes de sistema lineal de segundo orden en (30) son mayores que cero se asegura que $\dot{\theta} \to 0$ y $\dot{\theta} \to \theta_d$ conforme transcurre el tiempo. Al aumentar k_p se aumenta la rapidez en la respuesta y al aumentar k_v se aumenta el amortiguamiento en la respuesta.

2.4. Salida plana

Ahora considere la siguiente función que es conocida como la salida plana (**?**):

$$\boxed{y_f = M_{22}\alpha + \sigma\int_0^t \tilde{v}(r)dr, \quad \sigma = M_pC_z + \frac{2}{R}I_\phi, \quad \tilde{v} = v - v_d} \tag{31}$$

donde v_d es una constante y representa el valor deseado de la velocidad traslacional de avance del robot. Diferenciando con respecto al tiempo tres veces consecutivas a (31) y sustituyendo expresiones algebraicas en el modelo (22), (24) y con las definiciones en (28) y (31) se puede llegar a (**?**):

$$\dot{y}_f = M_{22}\dot{\alpha} + \sigma\tilde{v} \tag{32}$$

$$\ddot{y}_f = \sigma_1\alpha \tag{33}$$

$$\dddot{y}_f = \sigma_1\dot{\alpha} \tag{34}$$

2.5. Controlador para balanceo y avance traslacional

Empecemos definiendo el esfuerzo de control u como sigue (**?**):

$$\boxed{u = \tau_r + \tau_l} \tag{35}$$

Así usando (24), (33) y (35) se puede construir el diagrama de simulación analógica de la Figura 2:

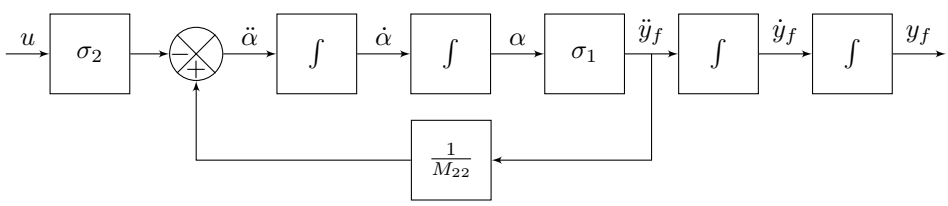

Figura 2: Diagrama de simulación analógica de (24) y (33).

Aplicando transformada de Laplace al diagrama de la Figura 2 se obtiene el diagrama de bloques siguiente (**?**):

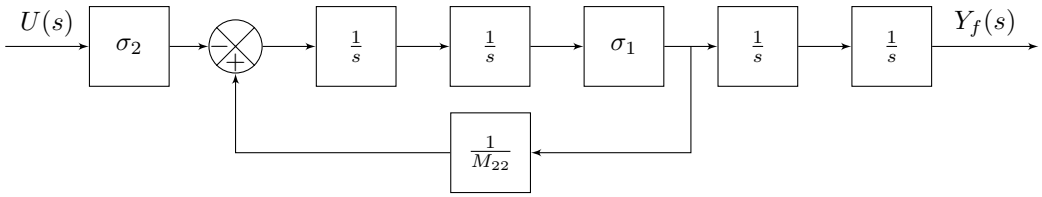

Figura 3: Diagrama de bloques de la Figura 2.

Aplicando álgebra de bloques al diagrama de la Figura 3 se obtiene el siguiente diagrama de bloques simplificado:

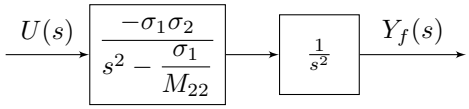

Figura 4: Diagrama de bloques simplificado de la Figura 3.

Dado que en la Figura 4 se tiene un sistema de cuarto orden, es posible asignar de manera arbitraria los cuatro polos de lazo cerrado si retroalimentamos y_f, $\dot{y}_f$, $\ddot{y}_f$ y $\dddot{y}_f$. Esto se puede conseguir con el sistema en lazo cerrado de la Figura 5 (**?**):

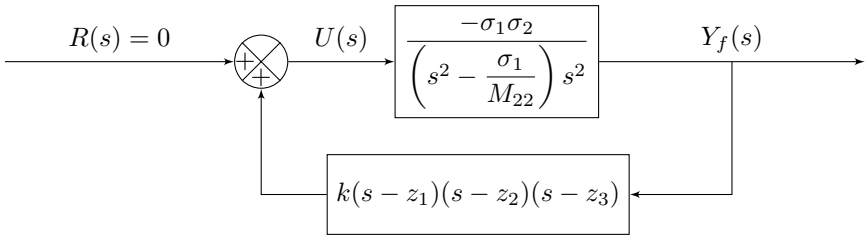

Figura 5: Diagrama de bloques del sistema en lazo cerrado.

Debido a que los ceros están localizados en z_1, z_2 y z_3 en la ruta de retroalimentación de la Figura 5 lo que da como resultado la siguiente retroalimentación de estado (**?**).

$$U(s) = k(s - z_1)(s - z_2)(s - z_3)Y_f(s) \tag{36}$$

que de manera polinomial está dada por:

$$U(s) = k(s^3 + a_2 s^2 + a_1 s + a_0)Y_f(s) \tag{37}$$

lo que implica:

$$s^3 + a_2 s^2 + a_1 s + a_0 = (s - z_1)(s - z_2)(s - z_3) \tag{38}$$

y usando transformada inversa de Laplace se tiene que:

$$u = k\ \dddot{y}_f + ka_2\ddot{y}_f + ka_1\dot{y}_f + ka_0 y_f \tag{39}$$

así sustituyendo (31), (32), (33) y (34) en (39) se obtiene:

$$\boxed{u = k_1\dot{\alpha} + k_2\alpha + k_3\tilde{v} + k_4\int_0^t \tilde{v}(r)dr} \tag{40}$$

donde:

$$k_1 = k(\sigma_1 + a_1 M_{22}) \tag{41}$$

$$k_2 = k(a_2\sigma_1 + a_0 M_{22}) \tag{42}$$

$$k_3 = ka_1\sigma \tag{43}$$

$$k_4 = ka_0\sigma \tag{44}$$

Una vez posicionados los ceros en z_1, z_2 y z_3 se puede encontrar el valor de k mediante el diagrama de la Figura 6 con el lugar geométrico de las raíces que permita asignar los polos deseados de lazo cerrado:

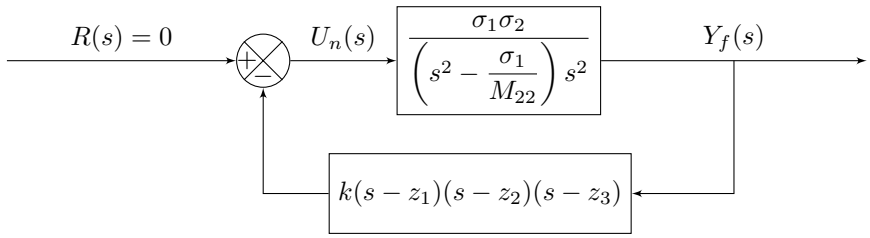

Figura 6: Diagrama de bloques del sistema en lazo cerrado con retroalimentación negativa, $U_n(s) = -U(s)$.

Es importante reflexionar sobre los efectos de las ganancias en los términos del controlador (40). Los dos primeros términos representan un controlador del tipo proporcional con retroalimentación de velocidad para balancear el robot, donde la ganancia k_1 pondera el efecto anticipativo (amortigua el comportamiento) mientras la ganancia k_2 pondera el efecto proporcional. Los últimos dos términos representan un controlador proporcional integral de velocidad traslacional del robot, donde la ganancia k_3 pondera el efecto proporcional mientras la ganancia k_4 pondera el efecto integral.

Debido a que el robot cuenta con dos actuadores (motor de la rueda derecha y motor de la rueda izquierda) es necesario que los esfuerzos de control τ_a en (29) y u en (35) comanden dichos actuadores, por lo que la relación que tienen con τ_r y τ_l está dada por (**?**):

$$\begin{bmatrix} \tau_a \\ u \end{bmatrix} = Q \begin{bmatrix} \tau_r \\ \tau_l \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \tag{45}$$

Como $det(Q) = 2$ entonces Q^{-1} existe y para calcular τ_r y τ_l en función de τ_a y u se requiere:

$$\begin{bmatrix} \tau_r \\ \tau_l \end{bmatrix} = Q^{-1} \begin{bmatrix} \tau_a \\ u \end{bmatrix}, \quad Q^{-1} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} \end{bmatrix} \tag{46}$$

Finalmente los pares aplicados a las ruedas son:

$$\tau_r = \frac{\tau_a}{2} + \frac{u}{2} \tag{47}$$

$$\tau_l = -\frac{\tau_a}{2} + \frac{u}{2} \tag{48}$$

3. Materiales y Métodos

3.1. Materiales

En el Cuadro 1 se muestran los materiales empleados para el desarrollo del PISDRSL.

Cuadro 1: Componentes electrónicos y mecánicos utilizados.

Material:	Cantidad:
Microcontrolador PIC18F4550	1
Sensor óptico reflexivo infrarrojo TCRT5000	2
Módulo bluetooth HC-05 5 $[k\Omega]$	1
Driver de Puente H L298N	1
Sensor MPU6050	1
Capacitor cerámico 100 $[nF]$	2
Capacitor cerámico 0.1 $[\mu$F$]$	1
Capacitor cerámico 15 $[pF]$	2
Capacitor cerámico 22 $[pF]$	2
Cristal de cuarzo 20 $[MHz]$	1
Resistencia 100 $[Omega]$	1
Resistencia 100 $[k\Omega]$	1
Resistencia 2.2 $[k\Omega]$	4
Interruptor de 2 polos	1
Batería LiPo Spard $2100[mAh]$, $11.1[V]$, 15C 3S RC	1
Espárragos de $\frac{3}{16}$ $[in]$	4
Cargador balanceador para batería	1
Motorreductor con *encoder* 12 $[V]$, 200 $[rpm]$	2
Llanta	2
Cople de motor	2
Placas de acrílico de 14.5 $[cm] \times 6.5$ $[cm]$ con un grosor de 5.5 $[mm]$	2
Fusible 2 $[A]$	1
Tornillos de cabeza hexagonal M3	18

Cuadro 2: Descripción de componentes.

Material	**Descripción general**
Batería RC	Batería de polímero de litio de alta descarga utilizada para alimentar circuitos y motores
Espárragos de $\frac{3}{16}$ $[in]$	Barras roscadas utilizadas para ensamblajes mecánicos y estructurales
Módulo puente H L298N	Módulo controlador de motores de corriente continua que permite cambiar la dirección y velocidad de los motores
Cargador balanceador para batería LiPo	Dispositivo que permite cargar baterías LiPo asegurando una distribución equilibrada de la carga en sus celdas
Motorreductor con *encoder* 12 $[V]$, 200 $[rpm]$	Motor de corriente continua con reducción de velocidad y sensor *encoder* para control de posición y velocidad
Llanta	Ruedas que permiten el desplazamiento del PISDRSL
Cople de motor	Acople mecánico utilizado para conectar el eje del motor con otros elementos mecánicos
Tarjeta MPU6050	Sensor de movimiento que integra un acelerómetro y giroscopio de 6 ejes para medición de inclinación y movimiento
Tarjeta Tcrt 5000	Sensor óptico de reflexión utilizado para la detección de línea
Microcontrolador PIC18F4550	Microcontrolador de 8 $[b]$ utilizado para el control del sistema
Placas de acrílico de 14.5 $[cm]$ $\times$ 6.5 $[cm]$ con un grosor de 5.5 $[mm]$	Acrílico utilizado para las placas del PISDRSL
Fusible 2 $[A]$	Dispositivo de protección que interrumpe el circuito en caso de sobre corriente
Capacitores cerámicos (22 $[pF]$, 0.1 $[\mu F]$)	Componentes electrónicos utilizados para el filtrado, estabilización de señales eléctricas y en circuitos de osciladores
Resistencias (2.2 $[k\Omega]$, 100 $[\Omega]$)	Resistores utilizados en circuitos electrónicos para limitar corriente, formar divisores de voltaje y controlar señales
Conectores JST	Conectores de uso común en sistemas electrónicos para conexiones seguras y rápidas
Interruptor de 2 polos	Dispositivo mecánico utilizado para controlar el encendido y apagado del circuito

En el Cuadro 3 se muestra el equipo de cómputo utilizado para el desarrollo del PISDRSL.

Cuadro 3: Equipo de cómputo utilizado.

Características:	Descripción:
Fabricante	Acer
ID del dispositivo	3C8E72CE-9F92-40F5-8A82-8D9E36929DB2
Procesador	11th Gen Intel(R) Core(TM) i7-11800H
Memoria RAM	8 $[GB]$, procesador x64
Tipo de sistema operativo	Sistema operativo de 64 $[b]$, procesador x64
Especificaciones de Windows	11 *Home Single Language*

En el Cuadro 4 se muestran los software utilizados para el desarrollo del PISDRSL.

Cuadro 4: El software empleado para el desarrollo de la práctica.

Software:	Versión:
CCS Compiler	Versión 5.083
MATLAB	R2021a

3.2. Diagrama de conexiones entre componentes

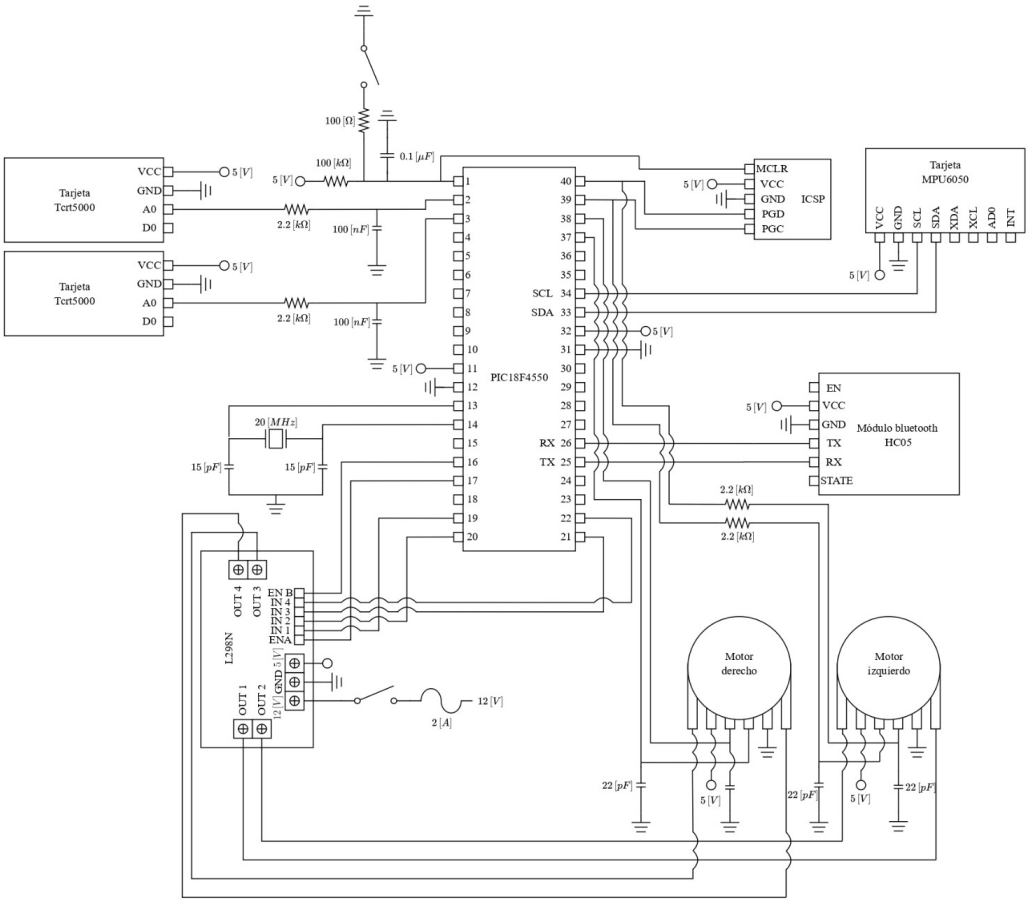

Figura 7: Conexión eléctrica de los componentes en el PISDRSL.

3.3. Metodología

La metodología utilizada para el desarrollo del PISDRSL se compone de diversas etapas:

1. **Recopilación de información:** Se realiza un análisis exhaustivo de los requerimientos de los componentes del PISDRSL, además del marco teórico relacionado con su comportamiento dinámico y los principios de control aplicables. Asimismo, se consideran mejoras potenciales y la retroalimentación proporcionada tanto por los integrantes del equipo como por el docente. En esta etapa también se identifican y adquieren los componentes electrónicos, materiales y herramientas necesarios para la construcción del sistema.

2. **Medición preliminar:** Se determinan las dimensiones y la disposición de los componentes electrónicos dentro de la estructura del PISDRSL para su ensamblaje. Además, se mide la masa de cada componente con el fin de distribuir adecuadamente la carga dentro de la estructura y calcular los momentos de inercia correspondientes, tanto del sistema completo como de sus elementos individuales. También se registra la velocidad de giro de los motores midiendo el número de vueltas por minuto, lo que permite estimar la velocidad angular máxima y los voltajes.

3. **Diseño estructural y electrónico:** Se desarrolla el diseño de la estructura del PISDRSL considerando sus especificaciones, restricciones mecánicas y distribución de peso. Paralelamente, se diseña la PCB, definiendo el trazado de las pistas de conexión eléctrica, la ubicación precisa de los componentes y la integración de los distintos subsistemas.

4. **Fabricación y ensamblaje:** Se procede a la manufactura de la PCB y al montaje de los componentes sobre la estructura del sistema. Para ello, se emplean elementos de sujeción como tornillos y tuercas, asegurando una fijación estable. Se realizan las conexiones eléctricas mediante cables con conectores tipo Molex, garantizando una interconexión correcta entre los distintos elementos del sistema.

5. **Verificación dimensional final:** Se obtienen mediciones precisas de la estructura final del PISDRSL, verificando la distancia entre los ejes de los motorreductores y los distintos componentes del sistema, asegurando la coherencia con el diseño previsto.

6. **Cálculo de parámetros:** Se determinan los parámetros físicos y matemáticos necesarios para modelar la dinámica del sistema, incluyendo los coeficientes de fricción, las inercias y la respuesta de los motores. También se calculan las ganancias del controlador que se implementará posteriormente.

7. **Implementación del control:** Se genera el algoritmo de control en el microcontrolador PIC18F4550 del PISDRSL, asegurando una programación correcta para la operación del sistema en tiempo real.

8. **Integración de comunicación inalámbrica:** Se establece la comunicación inalámbrica mediante *bluetooth*, permitiendo el monitoreo remoto del sistema.

9. **Sintonización y ajuste final:** Se realizan pruebas de funcionamiento para evaluar el desempeño del controlador. En esta etapa, se ajustan finamente las ganancias del control, mejorando la respuesta del sistema para que realice el recorrido especificado de manera precisa, estable y robusta ante cambios de iluminación.

4. Resultados y Discusión

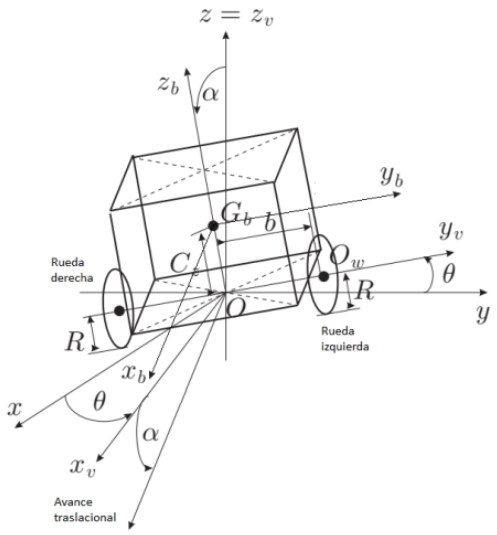

Figura 8: Imagen de oscilocopio con un muestreo de 10 $[ms]$ y la señal tiene una frecuencia de 45 $[Hz]$ sin filtro.

4.1. Aguilar Sarabia Diego Eugenio

En esta práctica se analizó la importancia del filtrado previo a la etapa de conversión analógico-digital para mitigar el fenómeno de *aliasing*. Al observar los resultados experimentales en el osciloscopio, se identificó que la señal senoidal presentaba componentes de alta frecuencia (ruido) que distorsionaban la información original. Para solucionar esto, se diseñó un filtro RC de primer orden con una frecuencia de corte teórica de 30 $[Hz]$. Debido a la disponibilidad de componentes, se empleó una resistencia comercial de 51 $[k\Omega]$, lo que ajustó la frecuencia de corte real a 31.2 $[Hz]$. Esta implementación permitió que el microcontrolador PIC18F4550 procesara una señal más limpia, cumpliendo con el objetivo de adquirir datos de manera fiel al limitar el ancho de banda de entrada conforme a los criterios de muestreo.

4.2. Fernández Rendón Anna Patricia

Se realizaron varios

4.3. Sánchez Alejo Frida Alexandra

4.4. Zeron Marin Luis Alejandro

4.5. Resultados y discusión del equipo

Para el desarrollo de esta práctica, se comprobó de forma experimental qué es y cómo se comporta el fenómeno de *aliasing*. A lo largo de todas las figuras, se puede observar que, a frecuencias menores que la frecuencia de Nyquist, la señal reconstruida puede distorsionarse, así como representarse de manera incorrecta, lo cual confirma la teoría de muestreo.

Sin embargo, este fenómeno puede mitigarse si se emplea un filtro *antialiasing*. Para esta práctica se utilizó un filtro RC de primer orden que, al ser calculado correctamente, permitió reducir los efectos del *aliasing*.

Cuando no se utilizaba el filtro, la señal presentaba distorsiones en su forma, asimetría e incluso saturación. Al implementar el filtro, se pudo observar una señal mucho más limpia y simétrica con respecto a la señal original.

5. Conclusiones

5.1. Aguilar Sarabia Diego Eugenio

5.2. Fernández Rendón Anna Patricia

5.3. Sánchez Alejo Frida Alexandra

5.4. Zeron Marin Luis Alejandro

5.5. Conclusiones del equipo

La materialización del prototipo PISDRSL, ejecutada sobre la arquitectura del microcontrolador PIC18F4550, ha constituido un ejercicio de ingeniería de alta exigencia. El despliegue, desde la concepción del modelo físico hasta la implementación de la placa de circuito impreso para la gestión electrónica, ha validado la capacidad técnica del equipo en la integración de sistemas de control.

El sustento teórico adquirido durante la asignatura de Control III resultó determinante para la decodificación y parametrización de las ganancias del algoritmo de control. Esta base académica permitió establecer una correlación directa entre los ajustes en el controlador y la respuesta dinámica del vehículo, optimizando así su comportamiento ante las variables del entorno.

La sinergia operativa del equipo fue fundamental para el éxito del proyecto. La aplicación de protocolos de comunicación asertiva permitió la detección temprana de anomalías, facilitando la ejecución de medidas correctivas inmediatas y evitando el escalamiento de los errores hacia la etapa de pruebas finales.

La planificación meticulosa y la validación sistemática de cada componente fueron los pilares que garantizaron el funcionamiento óptimo del PISDRSL sobre la pista. No obstante, la sintonización fina de los parámetros de control permanece como una línea de trabajo abierta; un refinamiento técnico en este aspecto es imperativo para maximizar el rendimiento y alcanzar mayores velocidades de desplazamiento.

Finalmente, la plataforma PIC18F4550 ha demostrado una robustez operativa notable para la gestión de tareas embebidas. Asimismo, la implementación de cubiertas para los sensores, concebidas para neutralizar la interferencia lumínica ambiental, representó una solución de diseño crítica para garantizar la estabilidad del sistema y la coherencia de las lecturas durante el recorrido en pista.

6. Anexos

6.1. Código Fuente del Sistema de Control de Equilibrio y Seguimiento de Línea en C (CCS Compiler)

```
#include <18F4550.h>
#device ADC=8
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#fuses HS,NOWDT,NOPROTECT, CPUDIV1,PLL5,NOLVP
#use delay(CLOCK=20M)
#use RS232(BAUD = 115200, XMIT = PIN_C6, RCV = PIN_C7, BITS = 8, PARITY = N)
#use I2C(MASTER, SDA=PIN_B0, SCL=PIN_B1, SLOW)

//Direcciones de los puertos
#byte porta = 0xf80
#byte portb = 0xf81
#byte portc = 0xf82
#byte portd = 0xf83
#byte porte = 0xf84

#bit  PD4   = 0xf83.4

//Referencias de los lazos
#define vd     0.1
#define alphad 0.0
#define thetad 0.0

//Ganancias lazo de inclinación (balance)
#define kpi 2.20//1.1961//1.94
#define kdi 0.16//0.1403//0.19

//Ganancias lazo de velocidad (avance)
#define kpv 1//0.8757//0.90
#define kiv 2//1.2474//2.57

//Ganancias lazo de orientación (seguimiento)
#define kpo 0.09//0.16//0.08
#define kdo 0.01//0.016//0.008

//Parámetros
#define Ts      0.01
#define ppr     12.0
#define pi_     3.141593
#define pi_s2   1.570796
#define Ra      3.0
#define NR      34.014
#define R       0.035
#define km      0.0008
#define b       0.09

#define calpha  0.145

#define tauM    0.1654  //par máximo en rueda [Nm]
#define alphaM  1.4  //1.57
#define omegaM  44.0 //70 rev en 30 s, 140 rev/min, 14.7 rad/s
#define uM      11
#define uNM     11

#define deg_2_rad          0.0174533
#define rad_2_deg          57.29578
#define accel_div_factor   16384.0
#define gyro_div_factor    131.0
#define accel_factor       0.0000610352
#define gyro_factor        0.0076336

#define c1 0.995//0.993

//Direcciones de registros del IMU
#define W_DATA         0xD0
#define R_DATA         0xD1
#define PWR_MGMT_1     0x6B
#define PWR_MGMT_2     0x6C
#define SMPRT_DIV      0x19
#define CONFIG_R       0x1A
#define GYRO_CONFIG    0x1B
#define ACCEL_CONFIG   0x1C
#define ACCEL_XOUT_H   0x3B
#define ACCEL_XOUT_L   0x3C
#define ACCEL_YOUT_H   0x3D
#define ACCEL_YOUT_L   0x3E
```

```
#define ACCEL_ZOUT_H   0x3F
#define ACCEL_ZOUT_L   0x40
#define GYRO_XOUT_H    0x43
#define GYRO_XOUT_L    0x44
#define GYRO_YOUT_H    0x45
#define GYRO_YOUT_L    0x46
#define GYRO_ZOUT_H    0x47
#define GYRO_ZOUT_L    0x48

//Registros del Timer
#byte TMR0H = 0xfd7 //TMR0H
#byte TMR0L = 0xfd6 //TMR0L
#byte T0CON = 0xfd5 //T0CON
#byte INTCON= 0xff2 //INTCON

//Variables a usar
signed int16 Ax,Gy;
int8 uWr,uWl;
signed int8 incr,incl;
float Sr,Sl;
int8 posr,posl;               //Posiciones o cuentas del encoder.
int8 puerto,AB,AB_1,CD,CD_1,auxAB,auxCD; // Variables auxiliares lectura de encoders.
int8 vdg,vg,thetag,alphag,omegalg,omegarg,ulg,urg;

signed int8 A_data_x[2];      //Lectura Ax del MPU
signed int8 A_data_y[2];      //Lectura Ay del MPU
signed int8 A_data_z[2];      //Lectura Az del MPU
signed int8 G_data_y[2];      //Lectura Gy del MPU

//Variables para filtro complementario y ángulos
float Xa=0.0,Yg=0.0,alpha=0.0;
float accelx=0.15,angulox=0.15,angulox_1=0.0,c2=1.0-c1;
//Variables auxiliares
float t=0.0,iTs=1.0/Ts,esc=pi_/(2.0*ppr*NR),escs=255.0/uM;
//Velocidades
float omegar=0,omegal=0;
//Voltajes a motores
float ur=0.0,ul=0.0;
float v=0.0,ei=0.0,ei_1=0.0,theta=0.0,eo=0.0,eop=0.0,taua=0.0,u=0.0;
float eip=0.0,ev=0.0,thetap=0.0,taur=0.0,taul=0.0;
float Rasnkm=Ra/(NR*km),nkm=NR*km,intev=0.0,v2tauM=2.0*tauM;
//pendientes para gráficas
float mv=255.0/(2.0*omegaM*R),mtheta=255.0/(2.0*0.3),malpha=255.0/(2.0*alphaM),momega=255.0/(2.0*omegaM),mu=255.0/(2.0*uM);

//Función para escribir al MPU6050
void MPU6050_write(int add, int data)
{
   i2c_start();
   i2c_write(W_DATA);
   i2c_write(add);
   i2c_write(data);
   i2c_stop();
}

//Función para leer datos del MPU
int16 MPU6050_read(int add)
{
   int retval;
   i2c_start();
   i2c_write(W_DATA);
   i2c_write(add);
   i2c_start();
   i2c_write(R_DATA);
   retval = i2c_read(0);
   i2c_stop();
   return retval;
}

//Función para iniciar el MPU
void MPU6050_init()
{
   MPU6050_write(PWR_MGMT_1, 0x80);
   delay_ms(100);
   MPU6050_write(PWR_MGMT_1, 0x00);
   delay_ms(100);
   MPU6050_write(CONFIG_R, 0x01);
   delay_ms(10);
   MPU6050_write(GYRO_CONFIG, 0x00);
}

//Función para leer el valor Ax del MPU
void MPU6050_lee_Ax()
{
   A_data_x[0] = MPU6050_read(ACCEL_XOUT_H);
```

```
   A_data_x[1] = MPU6050_read(ACCEL_XOUT_L);
}

//Función para leer el valor Ay del MPU
void MPU6050_lee_Ay()
{
   A_data_y[0] = MPU6050_read(ACCEL_YOUT_H);
   A_data_y[1] = MPU6050_read(ACCEL_YOUT_L);
}

//Función para leer el valor Az del MPU
void MPU6050_lee_Az()
{
   A_data_z[0] = MPU6050_read(ACCEL_ZOUT_H);
   A_data_z[1] = MPU6050_read(ACCEL_ZOUT_L);
}

//Función para leer el valor Gy del MPU
void MPU6050_lee_Gy()
{
   G_data_y[0] = MPU6050_read(GYRO_YOUT_H);
   G_data_y[1] = MPU6050_read(GYRO_YOUT_L);
}

//Interrupción para leer encoders en el puerto B pines B4-B7
#int_rb
void rb_isr()
{
    puerto = PORTB;
    AB =((puerto)&(0x30)) >> 4;
    auxAB=AB^AB_1;
    if(auxAB!=0)
       if(auxAB!=3)
       {
          if(((AB_1<<1)^AB)&(0x02))
             posr--;
          else
             posr++;
       }
    AB_1=AB;

    CD =((puerto)&(0xC0)) >> 6;
    auxCD=CD^CD_1;
    if(auxCD!=0)
       if(auxCD!=3)
       {
          if(((CD_1<<1)^CD)&(0x02))
             posl--;
          else
             posl++;
       }
    CD_1 = CD;
}

int main()
{
   //Configuración de pines como entradas o salidas.
   set_tris_a(0b11111111);
   set_tris_b(0b11110011);
   set_tris_c(0b01000000);
   set_tris_d(0b11100000);
   set_tris_e(0b11111111);
   //Iniciar el MPU
   MPU6050_init();   //Inicializa MPU6050

   T0CON=0xC7;       //Prescaler asignado a timer0 con relación 1:256
   TMR0L=0;
   PD4=0;
   //Inicializar variables para cuentas de encoder a 0
   AB=0;
   AB_1=0;
   //Inicio de cuentas encoder derecho a mitad del rango para que no desborde
   posr=127;
   // Inicializar variables para cuentas de encoder a 0
   CD=0;
   CD_1=0;
   //Inicio de cuentas encoder izquierdo a mitad del rango para que no desborde
   posl=127;

   //Configurar timer como PWM
   setup_timer_2(T2_DIV_BY_16,254,1);

   //Configurar pines 16 y 17 del 4550 como PWM
   setup_ccp1(CCP_PWM);          //configurando pwm
```

```
setup_ccp2(CCP_PWM);          //configurando pwm

setup_adc(ADC_CLOCK_INTERNAL);//Configurando el convertidor ADC con reloj interno
setup_adc_ports(AN0_TO_AN1);

//Se habilitan interrupciones
enable_interrupts(int_rb);
enable_interrupts(global);

while(true)
{
   PD4=1;   //Se prende bit de tiempo de ejecución
   //Se Leen sensores de línea
   set_adc_channel(0);
   delay_us(15);
   Sl=read_adc();

   set_adc_channel(1);
   delay_us(15);
   Sr=read_adc();

   //Se Leen los datos necesarios del MPU
   MPU6050_lee_Ax();
   MPU6050_lee_Gy();

   Ax=0;
   Ax=A_data_x[0];
   Ax=Ax<<8;
   Ax=Ax+A_data_x[1]; //Lectura cargada en variable Ax

   Gy=0;
   Gy=G_data_y[0];
   Gy=Gy<<8;
   Gy=Gy+G_data_y[1]; //Lectura cargada en variable Gy

   theta=Sr-Sl;
   if(theta>160.0)
      theta=160.0;
   if(theta<-160.0)
      theta=-160.0;
   theta=-0.3*theta/160.0;   //Error de seguimiento en radianes

   //Cálculo de velocidad derecha en radianes
   incr=127-posr;
   posr=127;
   omegar=incr*esc*iTs;
   //Cálculo de velocidad izquierda en radianes
   incl=127-posl;
   posl=127;
   omegal=incl*esc*iTs;

   //Los valores del MPU los paso a valores flotantes con sus respectivas escalas
   Xa=-Ax*accel_factor; //Xa en rango de -1 a 1
   Yg=Gy*gyro_factor;   //Yg en grados sobre segundo
   Yg=deg_2_rad*Yg;     //Yg en rad sobre segundo
   //Cálculo del filtro complementario
   accelx=Xa*pi_s2;  //Inclinación en rango de -pi/2 a pi/2 rad
   angulox=c1*(angulox_1+Yg*Ts)+c2*accelx;//Ecuación del filtro complementario
   angulox_1=angulox;//Respaldando valor pasado
   alpha=-angulox;
   alpha=alpha-calpha;//Compensación de alineación del MPU
   if(alpha>=alphaM)
      alpha=alphaM;
   if(alpha<=(-alphaM))
      alpha=-alphaM;

   //Cálculo de velocidad traslacional
   v=(omegar+omegal)*R/2.0;
   //Cálculo de theta punto
   thetap=(omegar-omegal)*R/(2.0*b);
   eop=-thetap;

   eo=thetad-theta;     //Se calcula theta tilde
   ei=alphad-alpha;
   eip=(ei-ei_1)*iTs;//Se calcula alpha punto
   ei_1=ei;
   ev=vd-v;                  //Se calcula v tilde
   if((intev<v2tauM)&&(intev>(-v2tauM))) //Integral de v tilde
      intev=intev+Ts*ev;
   else
   {
      if(intev>=v2tauM)
         intev=0.95*v2tauM;
      if(intev<=(-v2tauM))
```

```
      intev=-0.95*v2tauM;
}
//Esfuerzos de control
taua=(kdo*eop+kpo*eo)*2.0*b/R;
//taua=0;
u=-kpi*ei-kdi*eip-kpv*ev-kiv*intev;
if(u>=v2tauM)
   u=v2tauM;
if(u<=(-v2tauM))
   u=-v2tauM;

//Pares por rueda
taur=(taua+u)/2.0;  // par derecho
taul=(-taua+u)/2.0;  // par izquierdo

//Voltaje rueda izquierda
ul=taul*Rasnkm+nkm*omegal;
//ul=uNM*sin(0.314*t);
//ul=0.0;
//ul=5.0;
//ul=uNM;
if(ul>=uNM)
   ul=uNM;
if(ul<=(-uNM))
   ul=-uNM;
uWl=escs*fabs(ul);
if(ul<0)
{
   output_high(pin_d0);
   output_low(pin_d1);
}
else
{
   output_low(pin_d0);
   output_high(pin_d1);
}


//Voltaje rueda derecha
ur=taur*Rasnkm+nkm*omegar;
//ur=uNM*cos(0.314*t);
//ur=0.0;
//ur=-5.0;
//ur=uNM;
if(ur>=uNM)
   ur=uNM;
if(ur<=(-uNM))
   ur=-uNM;
uWr=escs*fabs(ur);
if(ur<0)
{
   output_high(pin_d2);
   output_low(pin_d3);
}
else
{
   output_low(pin_d2);
   output_high(pin_d3);
}

set_pwm1_duty(uWl); //Actualizando el valor del pwm rueda izquierda
set_pwm2_duty(uWr); //Actualizando el valor del pwm rueda derecha

//Se envían bytes al puerto serial para la gráfica experimental
putc(0xAA); //ID a la PC
vdg=(int8)(mv*vd+127);
putc(vdg);
vg=(int8)(mv*v+127);
putc(vg);
thetag=(int8)(mtheta*theta+127);
putc(thetag);
alphag=(int8)(malpha*alpha+127);
putc(alphag);
omegalg=(int8)(momega*omegal+127);
putc(omegalg);
omegarg=(int8)(momega*omegar+127);
putc(omegarg);
ulg=(int8)(mu*ul+127);
putc(ulg);
urg=(int8)(mu*ur+127);
putc(urg);

PD4=0;            //Se apaga bit de tiempo de ejecución
while(TMR0L<196); //Cada cuenta vale (4/FXtal)*256 seg
```

```
      t=t+Ts;
      TMR0L = 0;        //Reseteo contador del timer0
   }
return 0;
}
```